ORGNZ-05: Bucket-Level Access Control

> **Series:** ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 5 of 10 | **Created:** January 2026 | **Last Updated:** 11/07/2026

Overview

Bucket-level access control provides a straightforward way to isolate data by team, application, or business unit. By granting permissions to specific buckets, you can ensure teams only access data relevant to their responsibilities.

> **Canonical pattern (when bucket-level access fits):** Bucket-level access works well in specific scenarios — **compliance separation** (PCI/HIPAA), **retention isolation** (different retention requirements naturally produce distinct buckets), **retention-cost allocation** (billing a cost center for the data it retains — Dynatrace: *"You can use buckets to help allocate retention costs, so that the appropriate cost center is billed for their retained telemetry data."* Partition data (DT docs); other costs are allocated with `dt.cost.*` fields), or **hostile multi-tenancy** (strict cross-tenant isolation). For general team-scoped data access without one of those scenarios, `dt.security_context` **+ record-level permissions** (covered in **ORGNZ-06**) is usually the simpler choice — it scales beyond the custom-bucket limit and is mutable without re-routing ingest. See **IAM-04: Policy Authoring and Management** § 5 *Choosing Between Patterns* (Patterns 3 and 9 cover the bucket-based scenarios) for the canonical scenario gating. The patterns below teach bucket-level mechanics for when the scenario fits.

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS environment with Grail enabled
Permissions	`storage:bucket-definitions:read`, `storage:logs:read`
Knowledge	Completed ORGNZ-04 (Permissions in Grail)
Data	At least 1 hour of log data

Table of Contents

1. Bucket Permission Fundamentals
2. Policy Examples
3. Team Isolation Pattern
4. Default Buckets Policy
5. Verifying Bucket Access
6. Best Practices
7. When Bucket-Level Isn't Enough

Learning Objectives

By the end of this notebook, you will:
- Create IAM policies for bucket-level access
- Use bucket naming patterns in policies
- Implement team isolation using buckets
- Understand bucket permission best practices

Bucket Permission Fundamentals

Required Permissions

All bucket access policies must start with `storage:buckets:read`:

```
ALLOW storage:buckets:read WHERE &lt;condition&gt;;
ALLOW storage:logs:read;  // Then allow table access
```

Two-Step Pattern

Step	Purpose	Example
1. Bucket access	Define which buckets	`storage:buckets:read WHERE bucket-name = 'x'`
2. Table access	Define which data types	`storage:logs:read`, `storage:metrics:read`



Policy Examples

Example 1: Single Bucket Access

Grant a team access to their specific bucket:

```
{
  "name": "platform-team-logs-access",
  "description": "Platform team can access platform logs bucket",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name =
'team_platform_logs'; ALLOW storage:logs:read;",
  "tags": ["team:platform"]
}
```

Example 2: Multiple Buckets with IN Operator

Grant access to a list of specific buckets:

```
{
  "name": "finance-multi-bucket-access",
  "description": "Finance team can access multiple buckets",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name IN
('finance_logs', 'finance_events', 'finance_audit'); ALLOW storage:logs:read,
storage:events:read;",
  "tags": ["team:finance"]
}
```

Example 3: Bucket Name Pattern with STARTSWITH

Grant access to all buckets matching a prefix:

```
{
  "name": "prod-infrastructure-access",
  "description": "Access to all production infrastructure buckets",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name
STARTSWITH 'prod_infra_'; ALLOW storage:logs:read, storage:metrics:read,
storage:spans:read;",
  "tags": ["env:production"]
}
```

Example 4: Bucket Pattern with MATCH

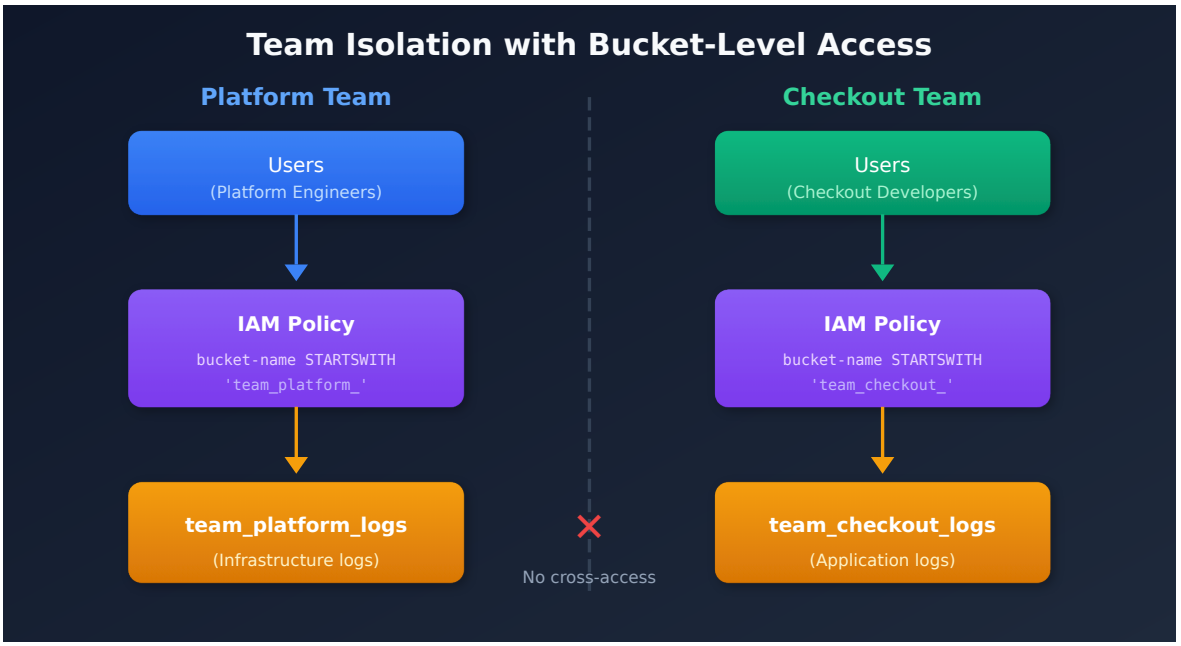
Use pattern matching for complex bucket names:

```
{
  "name": "database-team-access",
  "description": "Database team can access any database-related bucket",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name
MATCH ('*-database-*'); ALLOW storage:logs:read;",
  "tags": ["team:database"]
}
```

Team Isolation Pattern

> **When this approach fits:** team isolation via dedicated buckets is well-suited to scenarios where each team's data already needs its own bucket for compliance, retention, or hard-cost-attribution reasons. For routine per-team scoping without one of those reasons, `dt.security_context` **parameterization** (one policy template, bound per team — see **ORGNZ-06** and **IAM-04 Pattern 2**) is usually simpler and avoids the custom-bucket limit. The pattern below applies once you've decided buckets are the right tool for your team-isolation scenario.

Architecture



Implementation

Step 1: Create team buckets

```
team_platform_logs
team_checkout_logs
team_payments_logs
```

Step 2: Assign buckets in OpenPipeline

In the log pipeline's **Storage** stage, add one **Bucket assignment** processor per team. Each has a DQL matcher, and the stage applies the first processor that matches:

Matching condition (DQL)	Bucket
`matchesValue(dt.host_group.id, "platform-*")`	`team_platform_logs`
`k8s.namespace.name == "checkout"`	`team_checkout_logs`

Match on a field your logs actually carry. `service.name` is an OpenTelemetry resource attribute: on the validation tenant (10/12/2026) it was set on 7,521 of 784,813 logs in an hour (about 1%), against 765,062 (97%) for `k8s.namespace.name`. A matcher on `service.name` would leave most checkout logs in the default bucket, and nothing would error. Run the field-coverage check in **ORGNZ-10 § 3** before choosing.

See **ORGNZ-03 § Routing Data to Buckets** for how the Storage stage works and which matcher functions to use.

Step 3: Create IAM policies per team

```
// Platform team policy
ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH
"team_platform_";
ALLOW storage:logs:read;

// Checkout team policy
ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH
"team_checkout_";
ALLOW storage:logs:read;
```

Default Buckets Policy

Standard Default Access

For users who need access to all default buckets:

```
ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH "default_";
ALLOW storage:events:read, storage:logs:read, storage:metrics:read,
      storage:entities:read, storage:bizevents:read, storage:spans:read;
```

Default Plus Custom Buckets

```
ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH "default_";
ALLOW storage:buckets:read WHERE storage:bucket-name = "audit_logs_365d";
ALLOW storage:logs:read, storage:metrics:read, storage:spans:read;
```

Verifying Bucket Access

DQL: Verifying Bucket Access

Confirm you can read from the expected buckets:

```
// Verify bucket access — check data distribution across buckets you can read
fetch logs, from:-1h
| summarize count = count(), by:{dt.system.bucket}
| sort count desc
```

Best Practices

Practice	Rationale
Use consistent bucket naming	Enables pattern-based policies
Assign policies to groups, not users	Easier management, consistent access
Document all policies	Audit trail and governance
Test policies with sample users	Prevent access issues
Use STARTSWITH for team prefixes	Future-proof for new buckets
Combine with record-level for scale	Bucket limits (250 by default from SaaS 1.346; 80 before) may not suffice

When Bucket-Level Isn't Enough

Bucket-level access has limitations:

Limitation	Alternative
Custom-bucket limit (250 by default from SaaS 1.346; 80 before)	Use security context for finer granularity
Shared data needs	Use record-level permissions
Dynamic team membership	Use security context mapped to tags
Hiding sensitive fields	Use fieldsets (`storage:fieldsets:read`) — see ORGNZ-07

> **Bucket limit (SaaS 1.346):** the default custom-bucket limit per environment is **250** (previously 80). SaaS 1.346 began its staged tenant rollout on 08/25/2026; a tenant still below 1.346 keeps the 80 default. The increase-on-request path is unchanged: roughly one additional bucket per 10 GB of daily ingest, with requests above 1,000 buckets reviewed by Dynatrace support. Not every page has caught up: the upgrade best-practice pages (published 08/24/2026) still state 80, while Use Grail buckets to partition data (DT docs) now states *"you can configure up to 250 custom buckets by default"* (re-read 10/25/2026) and the newer How to organize your data stored in Grail (DT docs) (updated 09/22/2026) states *"The default limit per environment is 250 custom buckets."* (read 10/14/2026).

See **ORGNZ-06** and **ORGNZ-07** for advanced permission patterns.

Next Steps

Continue with the ORGNZ series:
- **ORGNZ-06**: Security Context

References

- Permissions in Grail
- IAM policy reference

This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.